Appello TAP del 19/02/2026 - Turno 2

Avete a disposizione **2 ore (2 ore e 30 minuti per chi ha DSA)**. La prova è composta da:

- **Quiz** su AulaWeb (11 punti)
- Esercizio di **programmazione** (11 punti)
- Esercizi di **testing** (6 + 5 = 11 punti)

La somma totale è **33 punti**, ma è necessario arrivare almeno a **5 punti su ciascuna delle tre parti** per passare l'esame.

Suggerimento: leggete **tutto** il testo prima di buttarvi a implementare "cose": la specifica dei test dell'esercizio 2 può aiutarvi a capire fin da subito cosa implementare (e cosa **NON** fare).

Per la consegna, caricate su AulaWeb la **solution "zippata"**, ovvero zippate la cartella che contiene il file `.sln` e i progetti, *dopo aver eseguito il Clean*. Codice che non compila o che non passa il `SanityCheck` (presente anche in versione `.txt` su AulaWeb) non verrà considerato. Dopo aver caricato la solution, per sicurezza, scaricate il vostro zip, scompattatelo in un'altra cartella e verificate che compili tutto.

Esercizio 1 (11 punti)

Implementare l'extension method `LimitConsecutive<T>` che, dati:

- una sequenza (*potenzialmente infinita*) di elementi di tipo `T`, `source`
- un intero che indica il numero massimo di ripetizioni consecutive ammesse, `maxRep`
- un confronto di uguaglianza `comparer`, di tipo `IEqualityComparer<T>?`

dovrà restituire, nell'ordine, gli elementi della sequenza `source` rimuovendo gli elementi **consecutivi** duplicati quando superano `maxRep`. Se `comparer==null` utilizzare il comparatore predefinito `EqualityComparer<T>.Default`.

L'unico metodo che vi serve, di quell'interfaccia, è:

```
public bool Equals(T? x, T? y);
```

che fa la cosa ovvia.

Quindi, per esempio, se `source==[10, 10, 10, 20, 10]` e `maxRep == 2`, allora `source.LimitConsecutive(2, null)` restituirà la sequenza `[10, 10, 20, 10]` (i primi tre 10 vengono ridotti a due, mentre l'ultimo 10 viene mantenuto perché non è consecutivo al primo blocco).

Se `maxRep <= 0`, lanciare `ArgumentOutOfRangeException` (**immediatamente**, senza che sia necessario enumerare la sequenza restituita).

Enumerare la sequenza `source` **una sola volta**.

```
[Fact]
public void SanityCheck() {
    IEnumerable<int> x = Array.Empty<int>().
                         LimitConsecutive(2, null);
    new List<string> { "a" }.LimitConsecutive(1,
                                 StringComparer.Ordinal);
}
```

Esercizio 2 (6 punti)

Implementare, usando xUnit, i seguenti test relativi all'esercizio 1:

1. Verificare che invocando il metodo con `maxRep==0` (o negativo) venga lanciata immediatamente l'eccezione `ArgumentOutOfRangeException`
2. Dato `source` l'array `[42]`, `maxRep==2` e `comparer==null`, verificare che il risultato sia la sequenza `[42]`
3. Dato `source` l'array `[1, 1, 1, 2, 2]`, `maxRep==2` e `comparer==null`, verificare che il risultato sia la sequenza `[1, 1, 2, 2]`
4. Dato `source` l'array `[1, 1, 3, 1, 1]`, `maxRep==1` e `comparer=null`, verificare che il risultato sia la sequenza `[1, 3, 1]`
5. Dato `source` la lista `["qui", "quo", "quo", "qua"]`, `maxRep==1` e `comparer==null`, verificare che il risultato sia la sequenza `["qui", "quo", "qua"]`
6. Dato `source` la lista `["a", "A", "a"]`, `maxRep == 1` e un `StringComparer.OrdinalIgnoreCase`, verificare che il risultato contenga solo un elemento.

Esercizio 3 (5 punti)

Per i punti 2, 3 e 4 dell'esercizio precedente, scrivere un corrispondente unit-test che verifichi che il metodo `Equals` del parametro `comparer`, venga utilizzato il numero "sensato" di volte, nell'enumerazione del risultato di `LimitConsecutive`.

Ovvero,

2. zero volte, non servono confronti
3. quattro volte, serve confrontare: 1 con 1, 1 con 1, 1 con 2, 2 con 2
4. quattro volte, serve confrontare: 1 con 1, 1 con 3, 3 con 1, 1 con 1

Suggerimento: basta un unico test *parametrico* per testare tutte queste condizioni (una per volta).